EXPENSE SPLITTER: A SPLITWISE-STYLE GROUP EXPENSE SETTLEMENT SYSTEM USING REACT, NODE.JS AND DESIGN PATTERNS

A PROJECT REPORT

Submitted by

PARIKSHIT (24BET10298)
DHRUV GOUNIYAL (24BET10295)
SAGAR MAKKAR (24BET10296)
MANAN (24BET10199)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY



Chandigarh University

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**EXPENSE SPLITTER: A SPLITWISE-STYLE GROUP EXPENSE SETTLEMENT SYSTEM USING REACT, NODE.JS AND DESIGN PATTERNS**" is the bonafide work of "PARIKSHIT (24BET10298), DHRUV GOUNIYAL (24BET10295), SAGAR MAKKAR (24BET10296) and MANAN (24BET10199)" who carried out the project work under my/our supervision.

SIGNATURE	**SIGNATURE**
Surinder Kaur	Suchi Sharma
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Information Technology	Assistant Professor
	Information Technology

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

List of Figures 4
List of Tables 4
CHAPTER 1. INTRODUCTION 6
1.1. Identification of Client / Need / Relevant Contemporary Issue 6
1.2. Identification of Problem 6
1.3. Identification of Tasks 7
1.4. Timeline 8
1.5. Organization of the Report 8
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY 9
2.1. Timeline of the Reported Problem 9
2.2. Existing Solutions 9
2.3. Bibliometric Analysis 10
2.4. Review Summary 10
2.5. Problem Definition 11
2.6. Goals/Objectives 11
CHAPTER 3. DESIGN FLOW/PROCESS 12
3.1. Evaluation & Selection of Specifications/Features 12
3.2. Design Constraints 12
3.3. Analysis of Features and Finalization Subject to Constraints 13
3.4. Design Flow 14
3.5. Design Selection 14
3.6. Implementation Plan/Methodology 15
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 19
4.1. Implementation of Solution 19
CHAPTER 5. CONCLUSION AND FUTURE WORK 29
5.1. Conclusion 29
5.2. Future Work 30
REFERENCES 31
APPENDIX 32
A. Design Checklist 32
B. Project at a Glance 32
C. Key Source Code Excerpts 33
D. Sample API Requests and Responses 34

List of Figures

Figure 1.1 Gantt chart of the project schedule 8
Figure 3.1 Block diagram of the layered architecture 15
Figure 3.2 Data model of the application (entities and relationships) 16
Figure 3.3 Class diagram of the Strategy, Observer and Singleton patterns 16
Figure 3.4 Flowchart of the transactional add-expense operation 17
Figure 4.1 Dashboard after loading the demonstration data (light theme) 22
Figure 4.2 PERCENTAGE split with the live split meter showing 10% unallocated 22
Figure 4.3 The server rejects the invalid split with 400 Bad Request 23
Figure 4.4 Expense list with per-member shares and the AuditLogger activity feed 23
Figure 4.5 Settle-up slip after a partial payment, with Settle buttons and payment history 24
Figure 4.6 Deleting a member who still has expenses returns 409 Conflict 24
Figure 4.7 Inline confirmation before deleting a group 24
Figure 4.8 Dashboard in the dark theme 25
Figure 4.9 Backend test suite run with npm test (node:test, 6 tests, 0 failures) 25
Figure 4.10 Newman run of the Postman collection (last two folders and summary) 27

List of Tables

Table 2.1 Timeline of key findings related to the problem 9
Table 2.2 Analysis of reviewed literature and solutions 10
Table 3.1 Evaluation of candidate features 12
Table 3.2 Comparison of alternative designs 14
Table 4.1 Tools used in the project 19
Table 4.2 Split strategies 20
Table 4.3 Balances for the demonstration scenario 21
Table 4.4 REST endpoints 21
Table 4.5 Functional test cases and results 26
Table 4.6 Greedy settlement compared with the exact minimum 28
Table A.1 Design checklist 32
Table A.2 Technical summary of Expense Splitter 32

ABSTRACT

Shared expenses are part of everyday student life: hostel rooms, flats, trips and group outings. UPI has made paying one another instant, but it does not answer the question that causes most disputes: who owes whom, and how much. Tracking this in chat messages or spreadsheets is error-prone, uneven splits are tedious to compute, and settling up pair by pair needs far more payments than necessary. This project presents Expense Splitter, a Splitwise-style full-stack web application that records group expenses, computes each member's net balance, proposes a short list of payments that settles the group and records those payments as they are made.

The system is written entirely in JavaScript: a React 18 single-page frontend built with Vite and a Node.js REST backend built with Express 4, in a layered controller–service–repository architecture. It applies three design patterns: Strategy for the three split methods (EQUAL, PERCENTAGE and EXACT), chosen at run time from the request; Observer for the side effects of adding an expense (an AuditLogger and a NotificationService); and Singleton for a ConfigService holding the currency and rounding rules. Adding an expense is one unit of work, so an invalid split rolls back the expense, its shares and its log together.

Net balances are computed with array methods (reduce and flatMap) and settlements with a greedy algorithm over two binary-heap priority queues, which needs at most $n - 1$ payments for n members. Money is rounded half-up to two decimals, and any paisa left over is assigned to one participant so that shares always add up exactly. A central error middleware returns clean JSON errors with the correct HTTP status codes.

The application was validated with a node:test suite and a Postman collection of 41 requests and 59 assertions run with Newman, all of which passed; it is hosted on GitHub and deployed on Vercel. Testing also showed that the greedy settlement, though never worse than $n - 1$ payments, is not always the true minimum (optimal in 100% of random four-member groups but 75.7% of eight-member groups), which is reported openly and addressed in future work.

Keywords: React, Node.js, Express, REST API, full-stack development, design patterns, Strategy, Observer, Singleton, greedy algorithm, expense sharing.

CHAPTER 1.
INTRODUCTION

1.1. Identification of Client / Need / Relevant Contemporary Issue

Shared spending is routine for students and young professionals. Hostel and PG roommates split rent, electricity and groceries; friends split trips, cab rides and restaurant bills; project teams split printing and travel costs. Each of these creates a running record of small debts between several people.

Digital payments have made the act of paying trivial. India's Unified Payments Interface (UPI), launched by NPCI in 2016, lets anyone transfer money instantly from a phone [1]. But payment apps answer "how do I pay?", not "how much do I owe, and to whom?". The bookkeeping that comes before the payment is still done in chat messages, notes or spreadsheets, which leads to forgotten expenses, arithmetic mistakes and awkward disputes between friends.

Dedicated apps such as Splitwise address this need [2], and their popularity shows that the need is real. They are, however, closed, account-based services that keep users' financial history on remote servers, and they do not let a student see how balances and settlements are actually computed.

There is also an academic need. The Full Stack Development (FSD) course asks students to build a complete web application — a frontend and a REST backend — that visibly applies component-based user interfaces, REST API design, validation and error handling, consistent data updates, modern JavaScript and object-oriented design patterns. An expense splitter fits well: its domain is familiar, yet it needs non-trivial relationships, validation, careful money arithmetic and a real algorithm.

The clients for this project are therefore **groups of students and young professionals** who share costs and want a transparent, private ledger, and **course instructors and students** who need a realistic, well-structured reference application.

1.2. Identification of Problem

Groups that share expenses face four connected problems:

- **Bookkeeping errors.** Records kept in chats or spreadsheets are easily lost, duplicated or mis-added, and nobody has a single trusted view of the group's balances.
- **Uneven splits.** Real expenses are not always equal: one person may have a larger room, skip a dish or pay a fixed amount. Computing percentage and exact splits by hand, and keeping the shares consistent with the total, is tedious.
- **Too many transactions.** If every debt is settled pair by pair, a group of n people can need up to $n(n-1)/2$ payments, many of which cancel each other out.
- **Money arithmetic.** Binary floating-point cannot represent most decimal amounts exactly [3], so naive code produces totals such as 999.99 for a Rs 1000 bill split three ways.

The broad problem is that there is no simple, transparent and correct way for a group to record shared expenses with different split rules, see who owes whom, and settle up with as few payments as possible.

1.3. Identification of Tasks

The work was divided into the following tasks, which also define the structure of this report:

1. **Problem identification and literature review** — study existing expense-sharing tools, the debt-settlement problem, money arithmetic, REST and enterprise design patterns (Chapters 1–2).
2. **Requirement and feature selection** — derive features and evaluate them against cost, privacy, ethical and technical constraints (Sections 3.1–3.3).
3. **Design** — compare alternative designs, select one, and model the architecture, the database and the expense-processing flow (Sections 3.4–3.6).
4. **Implementation** — build the entities and repositories, the Express REST API, the split strategies, the transactional expense flow, the balance and settlement engine, settle-up payments, observers, the configuration singleton, error handling and a React web interface (Chapter 4).
5. **Testing and validation** — write a node:test suite and a Postman collection with automated assertions, run it with Newman, and verify rollback, error codes and settlement results (Chapter 4).

6. **Documentation and deployment** — prepare this report, the README, a Postman test guide and a viva cheat-sheet; publish the code on GitHub and deploy the application on Vercel.

1.4. Timeline

The project was planned over eight weeks. Figure 1.1 shows the schedule as a Gantt chart; the split-strategy and transaction work and the settlement algorithm are highlighted because they carried the most technical risk.

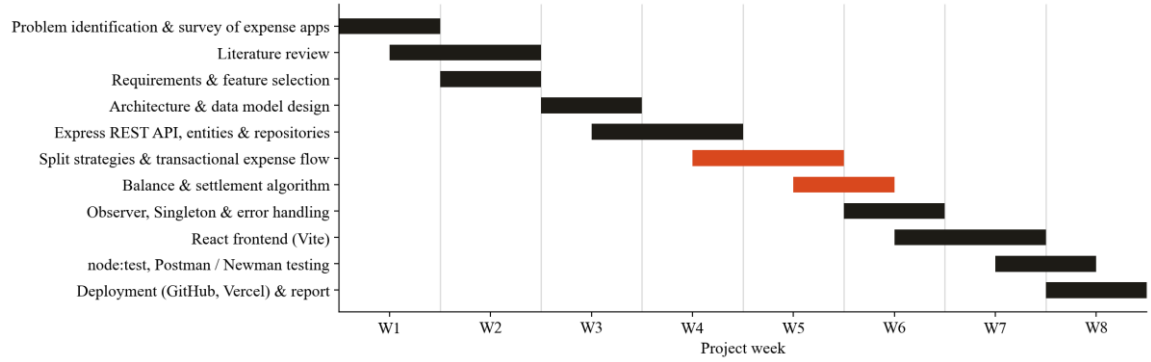


Figure 1.1 Gantt chart of the project schedule

1.5. Organization of the Report

- **Chapter 1 – Introduction** establishes the need, defines the problem, lists the tasks and presents the timeline.
- **Chapter 2 – Literature Review** traces how the problem has been studied, reviews existing solutions, analyses the literature and ends with a precise problem definition and measurable objectives.
- **Chapter 3 – Design Flow/Process** evaluates candidate features, lists design constraints, compares two alternative designs, selects one and presents the architecture, database design and algorithms.
- **Chapter 4 – Results Analysis and Validation** describes the implementation module by module, shows the working application and reports the outcome of the tests.
- **Chapter 5 – Conclusion and Future Work** summarises the outcomes, deviations from the plan and directions for extension.

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the Reported Problem

Table 2.1 Timeline of key findings related to the problem

Year	Milestone
1985	IEEE 754 standardises binary floating-point arithmetic, the number format used by JavaScript [4].
1991	Goldberg explains why binary floating-point cannot represent most decimal fractions exactly [3].
1994	Gamma et al. catalogue 23 object-oriented design patterns, including Strategy, Observer and Singleton [6].
1995	JavaScript is created at Netscape; it is standardised as ECMAScript in 1997 [5].
2000	Fielding defines the REST architectural style for web APIs [7].
2002	Fowler describes Service Layer, Repository and Unit of Work patterns [8].
2004	Verhoeff analyses how to settle multiple debts efficiently and shows that finding the fewest payments is a hard problem [11].
2009	Node.js brings event-driven JavaScript to the server [9].
2010	Express is released as a minimal web framework for Node.js [10].
2011	Splitwise launches, popularising app-based group expense sharing [2].
2013	React is open-sourced, introducing component-based, declarative user interfaces [12].
2016	NPCI launches UPI, making instant person-to-person payments widespread in India [1].

2.2. Existing Solutions

Existing approaches to managing shared expenses fall into five groups:

- **Dedicated expense-sharing apps (Splitwise, Tricount, Settle Up).** They track group expenses with equal and unequal splits and suggest simplified repayments. They are polished, but need an account and an internet connection, store financial history on the provider's servers and do not expose how results are calculated.
- **UPI payment apps with split features.** Some payment apps let a group split a single bill and send payment requests. This works for one bill, but is tied to one payment ecosystem and does not keep a multi-expense ledger with running balances.
- **Spreadsheets.** Flexible and free, but every group must build its own sheet, formulas break easily, nothing checks that shares add up, and settlement must be worked out by hand.

- **Chat messages and notes.** The most common approach in practice: zero setup, but no structure, no totals, no history and frequent disputes.
- **Accounting software.** Accurate double-entry bookkeeping, but designed for businesses and far too heavy for a group of friends.

2.3. Bibliometric Analysis

Table 2.2 Analysis of reviewed literature and solutions

Source / solution	Key features	Effectiveness	Drawbacks
Gamma et al. [6]	Catalogue of reusable patterns incl. Strategy, Observer, Singleton	Shared vocabulary; makes variation points explicit	Overuse adds indirection; Singleton can hide global state
Fowler [8]; Node.js [9]; Express [10]	Service layer, Repository, Unit of Work; event-driven server with middleware routing	Separates business rules from transport and storage; one language end to end	Needs discipline to keep layers apart; no built-in persistence
Fielding [7]	Resource URLs, uniform interface, stateless requests, status codes	Simple, language-neutral APIs	No transaction semantics across requests
Verhoeff [11]	Formal analysis of settling debts within a group	$n - 1$ payments always suffice	True minimum is hard to find; heuristics needed
Goldberg [3]; IEEE 754 [4]	Analysis of binary floating-point error	Explains rounding errors in money calculations	double needs explicit rounding for money
Splitwise and similar apps [2]	Group ledger, uneven splits, debt simplification	Widely used and convenient	Accounts, remote data, closed algorithms

2.4. Review Summary

The literature establishes four points that shape this project. First, a layered design with a service layer and repositories keeps business rules testable and separate from HTTP and storage [8], and Express's router and middleware model maps naturally onto that layering [10]. Second, Strategy and Observer are the standard ways to make "many ways to split" and "many reactions to an event" extensible without long if-else chains [6]. Third, money must be handled with explicit rounding because binary floating-point, which JavaScript uses for every number, is inexact [3], [4]. Fourth, settling a group's debts never needs more than $n - 1$ payments, but guaranteeing the absolute minimum is computationally hard [11], so a fast greedy heuristic is the practical choice for an interactive system.

Expense Splitter responds directly: it exposes a REST API [7] built with Node.js and Express and consumed by a React interface [12], uses Strategy, Observer and Singleton where they fit naturally, rounds money explicitly to two decimals, and settles debts with a two-heap greedy algorithm whose behaviour is measured and documented in Chapter 4.

2.5. Problem Definition

What is to be done: build a full-stack web application through which a group can register members, record expenses split by EQUAL, PERCENTAGE or EXACT rules, view each member's net balance, obtain a short list of payments that settles the group, record those payments, and delete expenses, payments, users and groups safely.

How it is to be done: a React frontend (Vite) calling a Node.js/Express REST API; a controller–service–repository architecture; the Strategy, Observer and Singleton patterns; all-or-nothing expense creation; a priority-queue greedy settlement; central error middleware; testing with node:test and Postman/Newman; deployment on Vercel.

What is not to be done: no real money movement or payment-gateway integration; no user authentication or multi-tenant security (a single trusted group in a lab setting); no currency conversion; no native mobile app; and no claim that the settlement is always the mathematical minimum.

2.6. Goals/Objectives

1. Provide REST endpoints to create users and groups, add members, add, list, filter and delete expenses, view balances and settlements, record and undo payments, and delete users and groups, each returning the correct HTTP status code.
2. Implement EQUAL, PERCENTAGE and EXACT splitting as interchangeable strategy classes selected at run time, with validation that percentages total 100 and exact amounts total the expense.
3. Guarantee atomicity: an expense, its shares and its activity log are saved together or not at all.
4. Compute net balances that always sum to zero, correct to two decimal places.
5. Produce a settlement that needs at most n − 1 payments for n members with non-zero balances.
6. Notify independent listeners (audit log, notification) whenever an expense is added or deleted, without coupling them to the expense service.
7. Validate the system with a node:test suite and a Postman collection of at least 50 assertions covering success and error paths, and deploy it on the web.

CHAPTER 3.
DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

Table 3.1 lists the features suggested by the literature and existing solutions, with a judgement of whether each is required in Expense Splitter.

Table 3.1 Evaluation of candidate features

Feature	Justification	Decision
Users, groups and membership	Core domain; many-to-many relationship	Required
Expenses with payer and participants	Core domain	Required
EQUAL / PERCENTAGE / EXACT splits	Real expenses are often uneven	Required (Strategy pattern)
Net balance per member	One trusted view of who owes whom	Required (array reduce)
Simplified settlement	Avoid $n(n-1)/2$ pairwise payments [11]	Required (greedy, $\leq n-1$)
Record settle-up payments	Balances must reflect money already paid back	Required (partial payments, undo)
Activity log and notifications	Transparency and audit trail	Required (Observer)
Atomic expense creation	No half-saved expenses	Required (transactional())
Clean error responses	REST conventions [7]	Required (error middleware)
Delete expenses, users, groups	Correct mistakes and clean up	Required (with integrity rules)
React web interface	Usable front end for a full-stack project	Required (React + Vite)
User accounts / login	Needed for public deployment	Deferred (lab scope)
Real UPI payments	Convenience	Rejected (regulation, security)
Multi-currency, receipts, recurring bills	Convenience	Deferred to future work

3.2. Design Constraints

3.2.1. Standards and Constraints Considered

- **Economic / cost:** only free, open-source software (Node.js, Express, React, Vite, Postman free tier); runs on a student laptop with a single npm command, and the deployment uses the free tiers of GitHub and Vercel.
- **Regulations / privacy:** the system only records who paid what; it never moves money and stores no bank, card or UPI details. Nothing beyond a name and an e-mail address is collected, and data is kept in the server's memory only.

- **Ethical:** money calculations must be transparent and fair. Every share is stored and visible, rounding remainders follow a documented rule, payments cannot exceed what is owed, and a user who still appears in expenses cannot be deleted silently (409 Conflict), because that would change other members' balances.
- **Safety / reliability:** adding an expense is a single unit of work that is rolled back on failure; input is validated before it is accepted; wrong IDs, amounts, split types and malformed JSON produce clear messages instead of server errors.
- **Professional standards:** REST conventions for URLs, methods and status codes [7]; layered architecture [8]; ES modules and one responsibility per file; the React interface uses function components and hooks [12]; no TypeScript or extra libraries, so that every line is visible to beginners.
- **Manufacturability / portability:** pure JavaScript on Node.js; the same code runs on Windows, macOS and Linux, locally with npm start or as a serverless function on Vercel; storage is behind repositories, so a database can be added without changing the services.
- **Accessibility:** the web interface supports keyboard use with visible focus, offers light and dark themes, and never relies on colour alone (balances are signed numbers with labels).

3.3. Analysis of Features and Finalization Subject to Constraints

- **Removed:** real UPI payment integration and user login (conflict with lab scope, regulation and security).
- **Modified:** balances were first planned as a stored value updated on every expense; they are instead computed on demand from expenses, shares and payments with reduce, so they can never drift out of sync after deletes.
- **Modified:** JavaScript has only one floating-point number type, so every computed amount passes through ConfigService.round() (half-up on the decimal text, so 1.005 becomes 1.01), and split remainders are assigned to one participant so shares always sum exactly.
- **Modified:** data is kept in in-memory repositories (one table per entity, with IDENTITY-style ids) so the app runs with zero setup; the repositories hide storage, so MongoDB or PostgreSQL can replace them later.

- **Modified:** deletion rules — deleting a group cascades to its expenses, shares, payments and log inside one transaction; deleting a user who still appears in expenses or payments is refused with 409 Conflict.
- **Added:** settle-up payments with partial amounts and undo, a React interface with a live API console and a split meter, a Postman collection that cleans up after itself, and deployment on Vercel.

3.4. Design Flow

Two complete alternative designs were considered.

Design A — Plain Node.js http server with conditional logic. One request handler parses every URL by hand, the split method is chosen with an if-else chain, balances are stored and updated manually, errors are caught in every branch, and the page is built from HTML strings.

Design B — Layered Express backend with a React frontend (selected). Routers translate HTTP into method calls; services hold the business rules; repositories store data; split methods are Strategy classes; side effects are Observer listeners; expense creation runs in a transactional() unit of work; one error middleware maps errors; and the interface is built from React components.

3.5. Design Selection

Table 3.2 Comparison of alternative designs

Criterion	Design A: Plain Node http	Design B: Express + React
Boilerplate	Manual URL and JSON parsing	express.json(), Router, params
Adding a new split type	Edit the if-else chain	Add one SplitStrategy subclass
Consistency on failure	Manual clean-up	transactional() rolls back
Error handling	try-catch in every branch	One error middleware
Consistency of balances	Stored value can drift	Computed from source data
Testability	Logic mixed with HTTP and HTML	Services tested with node:test
Fit with FSD syllabus	Partial (no UI framework)	Complete: React, Express, REST
Learning curve	Lower at first	Higher, mirrors industry practice

Design B was selected. It satisfies every required feature in Table 3.1, keeps business rules isolated and testable, and demonstrates every concept the experiment asks for.

3.6. Implementation Plan/Methodology

Figure 3.1 shows the layered architecture. Clients (the React interface and Postman) call the Express controllers; controllers delegate to services; services use the Strategy, Observer and Singleton components and the repositories, which keep the data in memory.

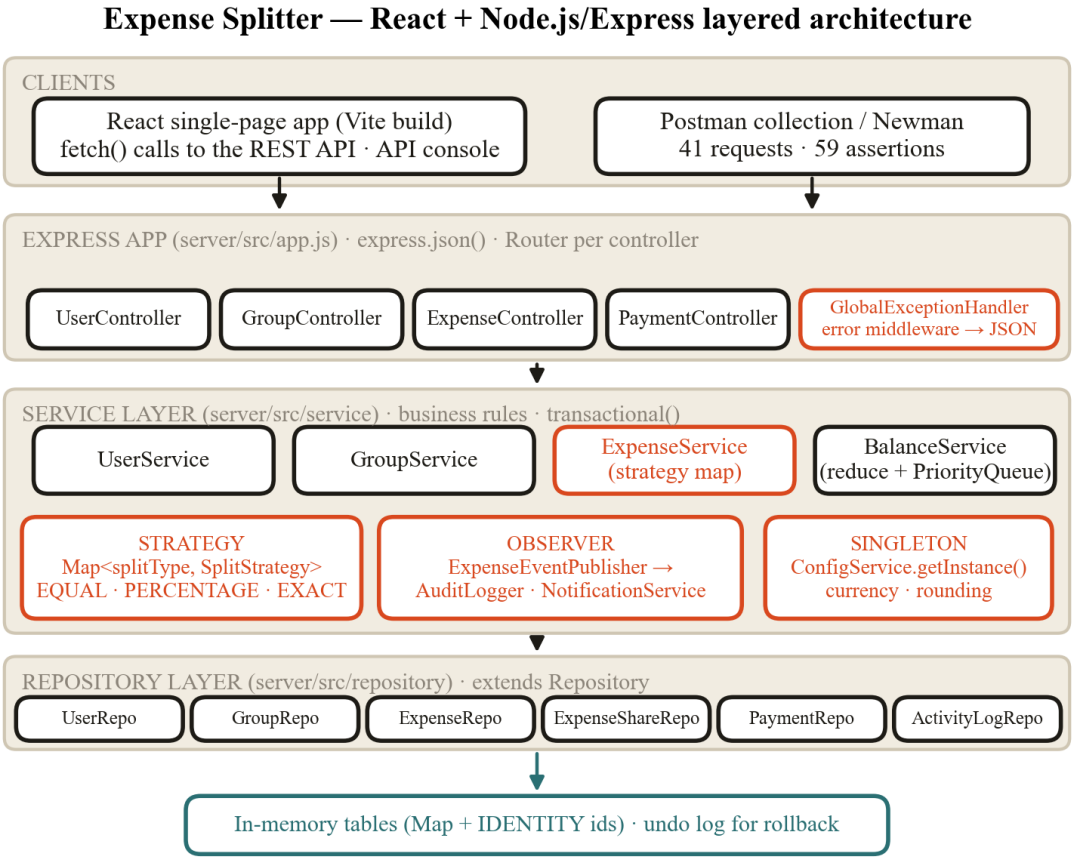
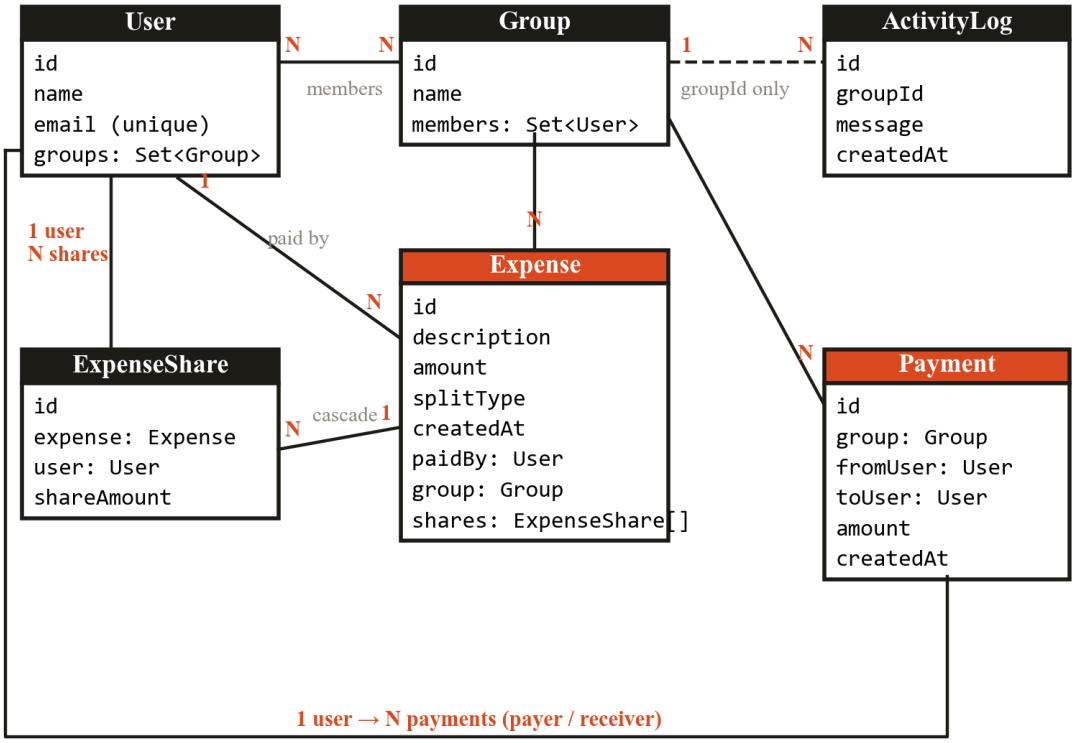


Figure 3.1 Block diagram of the layered architecture

Figure 3.2 shows the data model. User and Group are linked many-to-many through their member sets; ExpenseShare stores how much each person owes for each expense; Payment stores settle-up payments between members. Each repository keeps one entity type in its own table with IDENTITY-style ids, and toJSON() hides back-references so that the JSON never loops.



Object references in memory · toJSON() hides back-references (User.groups, ExpenseShare.expense, *.group → groupId)

Figure 3.2 Data model of the application (entities and relationships)

Figure 3.3 shows how the three design patterns are realised as classes, and Figure 3.4 the complete flow of adding an expense, including the points where the request is rejected and where the transaction is rolled back.

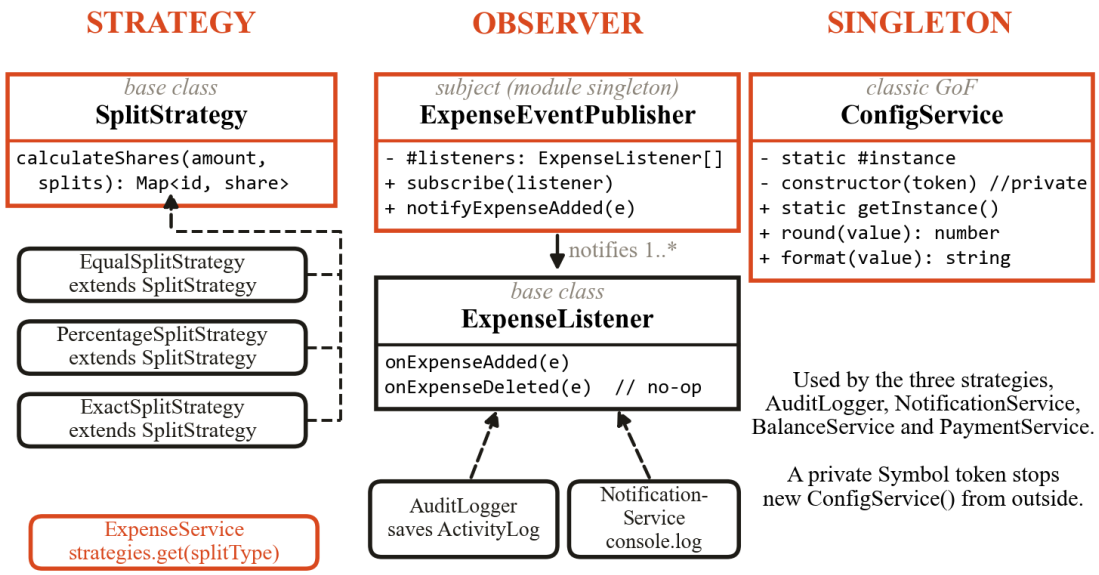


Figure 3.3 Class diagram of the Strategy, Observer and Singleton patterns

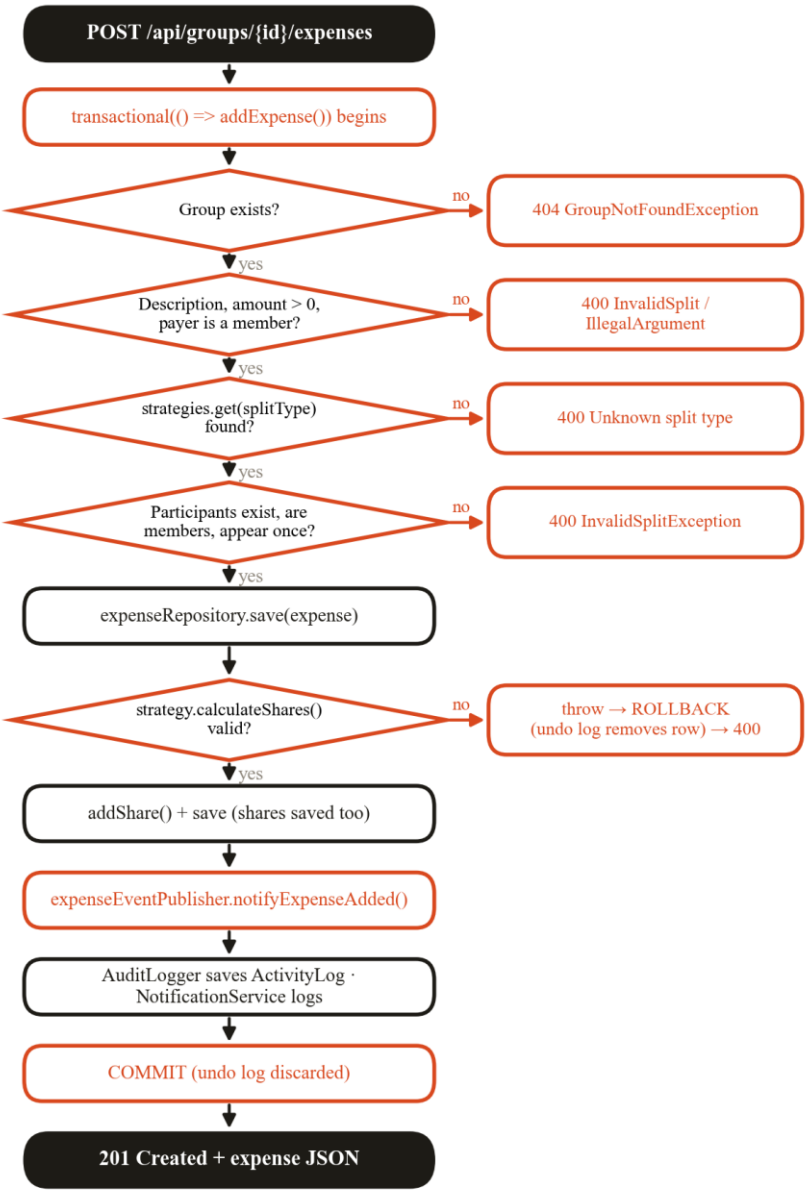


Figure 3.4 Flowchart of the transactional add-expense operation

Algorithm: computing balances and settlements

1. Load all expenses of the group with their shares, and all recorded payments.
2. paid[u] = total amount of expenses that member u paid; owed[u] = total of u's shares (Array.reduce into a Map, using flatMap over the shares).
3. sent[u] and received[u] = totals of settle-up payments made and received by u.
4. For every member, net = round(paid − owed + sent − received). Positive means the member gets money back, negative means the member owes, and $|net| \leq 0.01$ means settled. The nets always sum to zero.

5. Put every member with net > 0 into a max-heap of creditors and every member with net < 0 into a max-heap of debtors keyed by |net| (a binary-heap PriorityQueue with an arrow-function comparator [17]).
6. While both heaps are non-empty, remove the largest creditor C and the largest debtor D; D pays C the amount x = min(C, D), recorded as “D pays C Rs x”.
7. Subtract x from both; whoever still has a non-zero amount is pushed back into its heap.
8. Each iteration settles at least one member completely, so the loop ends after at most n − 1 payments, with a cost of O(n log n).

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of Solution

4.1.1. Tools and technologies

Table 4.1 Tools used in the project

Tool	Use
Node.js 24 (works on 18+)	Backend runtime: ES modules, classes, private fields, built-in test runner
Express 4.21	Routing, JSON body parsing, route parameters, error-handling middleware
React 18.3	Component-based single-page interface with hooks (useState, useEffect, useRef, useCallback)
Vite 5.4	Development server with an /api proxy and the production build of the React app
JavaScript (ES2022)	One language for client and server: arrow functions, array methods, Map and Set
node:test, node:assert	Unit and HTTP integration tests run with npm test
Postman, Newman 6	API testing with automated assertions; command-line runs
Playwright	Automated browser screenshots of the interface for this report
Git, GitHub, Vercel	Version control, code hosting and serverless deployment
VS Code, Claude Code	Development environment and AI-assisted development
Matplotlib, Microsoft Word	Diagrams and report preparation

4.1.2. Code organisation

The backend consists of 34 JavaScript modules (about 1,420 lines including comments) in ten folders under server/src, and one test file. **entity:** User, Group, Expense, ExpenseShare, Payment, ActivityLog. **repository:** a generic Repository with transactional() and six repositories. **dto:** request parsers that coerce and validate JSON bodies. **strategy:** SplitStrategy and three subclasses. **observer:** ExpenseListener, ExpenseEventPublisher, AuditLogger, NotificationService. **config:** ConfigService. **service:** User, Group, Expense, Balance and Payment services and a PriorityQueue. **controller:** four Express routers and id parsing. **exception:** error classes and the GlobalExceptionHandler. The frontend (client/src) has eight modules (about 820 lines): App.jsx, api.js, format.js and the components PeopleCard, GroupsCard, ExpenseForm, GroupPanel and ApiConsole, plus a 300-line stylesheet.

4.1.3. Split strategies

Each split type is a subclass of SplitStrategy registered in a Map under its type name, so ExpenseService can look it up with strategies.get(splitType). Table 4.2 summarises the rules.

Table 4.2 Split strategies

Split type	Value sent per person	Rule	Validation	Rounding
EQUAL	None	amount ÷ participants	At least one participant	Leftover paisa to the first person
PERCENTAGE	Percent	amount × percent ÷ 100	Values ≥ 0; total = 100 (±0.01)	Leftover to the last person
EXACT	Rupees	Value as given	Values ≥ 0; total = amount (±0.01)	Each value to 2 decimals

For example, Rs 1000 split equally among three people gives 333.34, 333.33 and 333.33, so the shares always add up to exactly the bill.

4.1.4. Transaction management and rollback

ExpenseService.addExpense() runs inside transactional(), a small unit-of-work helper in the repository layer. While it runs, every insert and delete is written to an undo log. The method first saves the expense and only then asks the strategy for the shares. If the split is invalid, the strategy throws InvalidSplitException, transactional() replays the undo log in reverse and rethrows, so the expense row disappears again. The AuditLogger runs inside the same unit of work, so the log entry is saved or discarded together with the expense. As with a database IDENTITY column, the id taken by a rolled-back expense is not reused: an expense rejected for percentages totalling 90 consumed ID 4 and the next successful expense received ID 5. The test suite checks that the number of expenses is unchanged after a rejected split.

4.1.5. Balance and settlement engine

Table 4.3 shows the balances for the demonstration scenario: a Rs 1200 dinner paid by Aman split equally, a Rs 1000 cab paid by Bhavya split 25/25/30/20 percent, and Rs 1000 of movie tickets paid by Chirag split exactly as 250/300/200/250.

Table 4.3 Balances for the demonstration scenario

Member	Paid (Rs)	Share owed (Rs)	Net balance (Rs)
Aman	1200	300 + 250 + 250 = 800	+400
Bhavya	1000	300 + 250 + 300 = 850	+150
Chirag	1000	300 + 300 + 200 = 800	+200
Diya	0	300 + 200 + 250 = 750	−750

The settlement algorithm returns three payments — “Diya pays Aman Rs 400.00”, “Diya pays Chirag Rs 200.00” and “Diya pays Bhavya Rs 150.00” — instead of the up to six pairwise payments a four-person group could need. When a member records a payment, for example Diya paying Aman Rs 250, the payment enters the balance formula, Diya’s debt falls to Rs 500, and the next settlement suggests only the remaining Rs 150 to Aman (Figure 4.5).

4.1.6. REST API

Table 4.4 REST endpoints

Method	Endpoint	Purpose	Success	Errors
POST	/api/users	Create a user	201	400
GET	/api/users, /api/users/{id}	List users / get one	200	404
DELETE	/api/users/{id}	Delete a user	204	404, 409
POST	/api/groups	Create a group	201	400
GET	/api/groups, /api/groups/{id}	List groups / get one	200	404
DELETE	/api/groups/{id}	Delete a group and its data	204	404
POST	/api/groups/{g}/members/{u}	Add a member	200	400, 404
POST	/api/groups/{g}/expenses	Add an expense	201	400, 404
GET	/api/groups/{g}/expenses?userId=	List / filter expenses	200	404
DELETE	/api/expenses/{id}	Delete an expense	204	404
GET	/api/groups/{g}/balances	Net balance per member	200	404
GET	/api/groups/{g}/settlements	Suggested payments	200	404
POST	/api/groups/{g}/payments	Record a settle-up payment	201	400, 404
GET	/api/groups/{g}/payments	List payments	200	404
DELETE	/api/payments/{id}	Undo a payment	204	404
GET	/api/groups/{g}/activity	Activity log (observer)	200	404

All errors share one JSON format produced by the GlobalExceptionHandler middleware, for example {"status": 400, "error": "Bad Request", "message": "Percentages must add up to 100, but got 90"} together with a timestamp. Wrong HTTP methods return 405, malformed IDs and broken JSON return 400, and unknown URLs return 404 in the same format.

4.1.7. User interface

The React interface is built with Vite and served by the same Express server at http://localhost:8080 (and by Vercel in production). App.jsx keeps the shared state and loads data through a small api() helper based on fetch(); components such as ExpenseForm and GroupPanel keep their own form state with useState. It uses a ledger-style design with light and dark themes, and a console strip at the bottom shows every request with its method, URL, status code and JSON body. Figures 4.1 to 4.8 show the main screens; Figures 4.9 and 4.10 show the automated test runs.

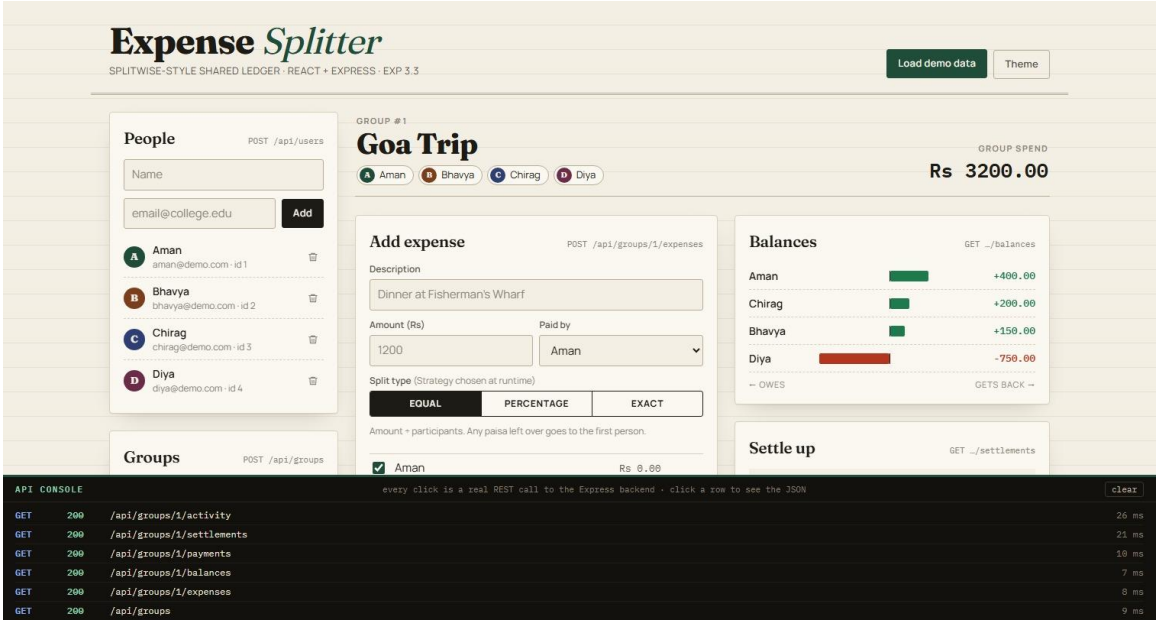


Figure 4.1 Dashboard after loading the demonstration data (light theme)

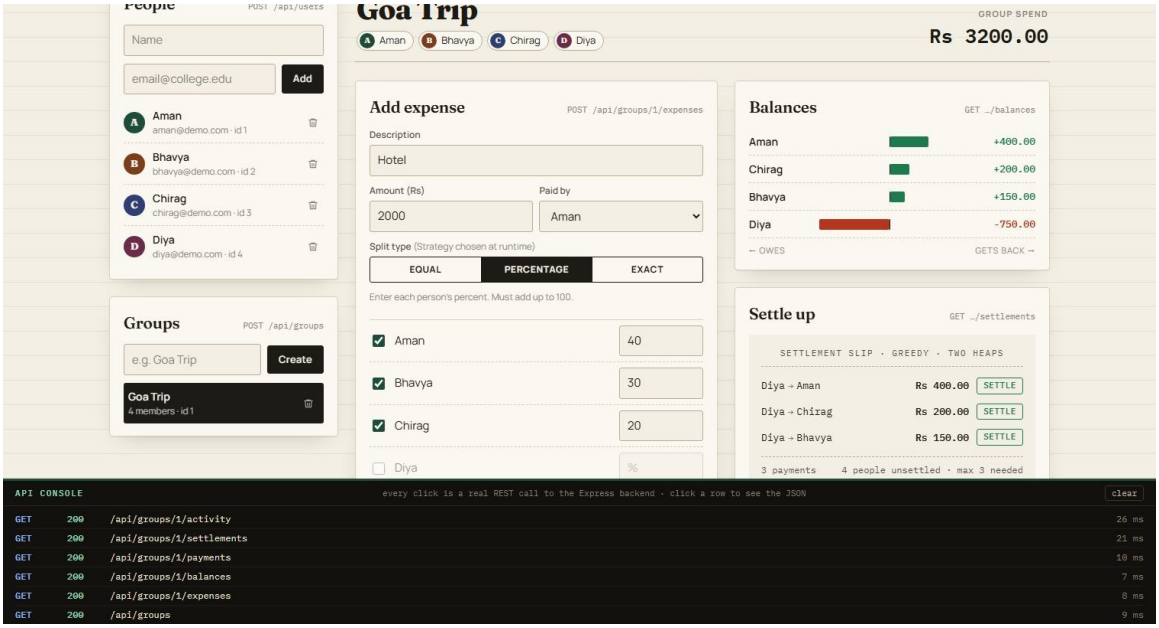


Figure 4.2 PERCENTAGE split with the live split meter showing 10% unallocated

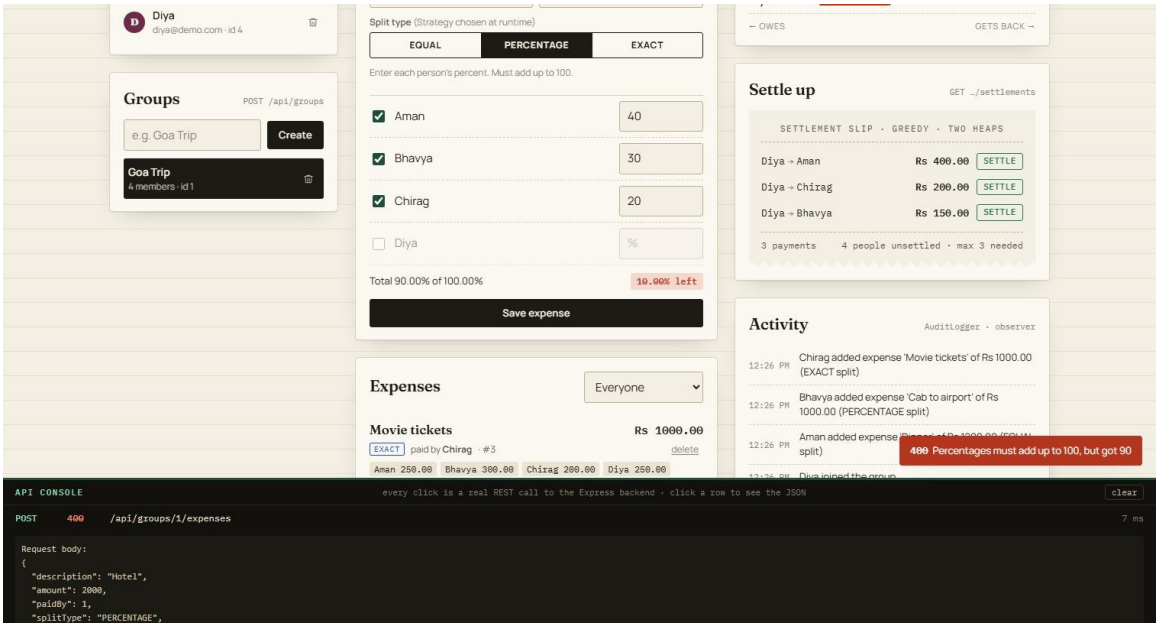


Figure 4.3 The server rejects the invalid split with 400 Bad Request

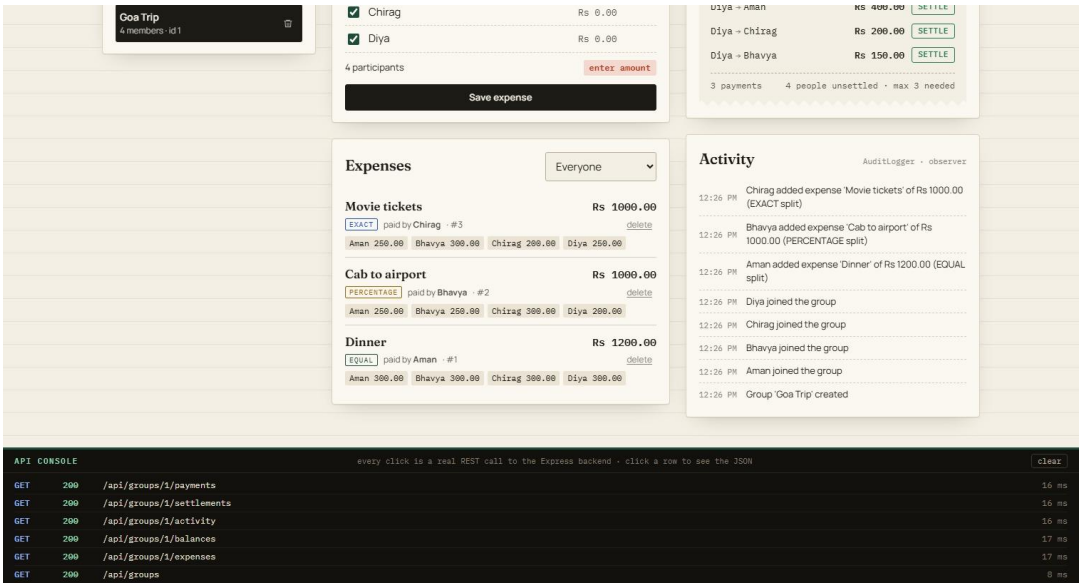


Figure 4.4 Expense list with per-member shares and the AuditLogger activity feed

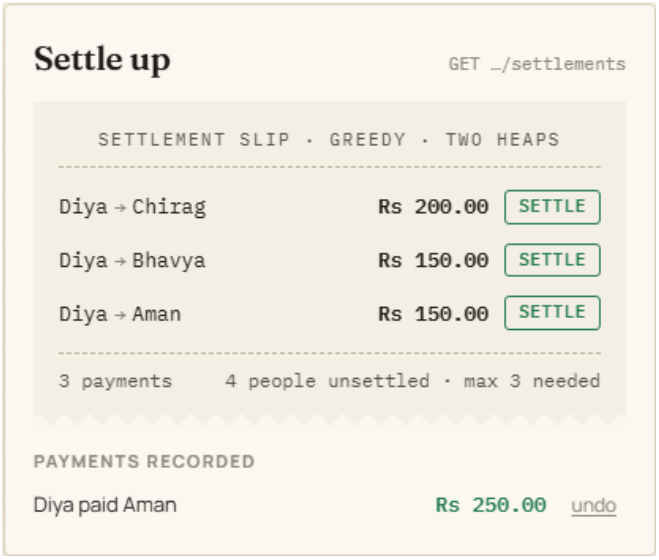


Figure 4.5 Settle-up slip after a partial payment, with Settle buttons and payment history

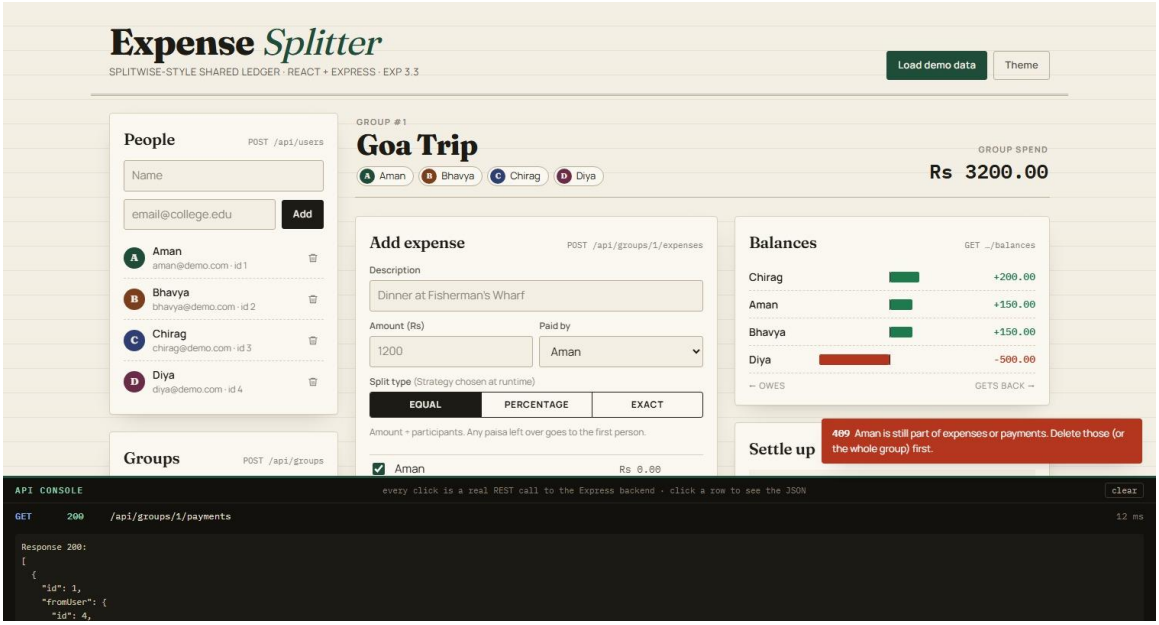


Figure 4.6 Deleting a member who still has expenses returns 409 Conflict

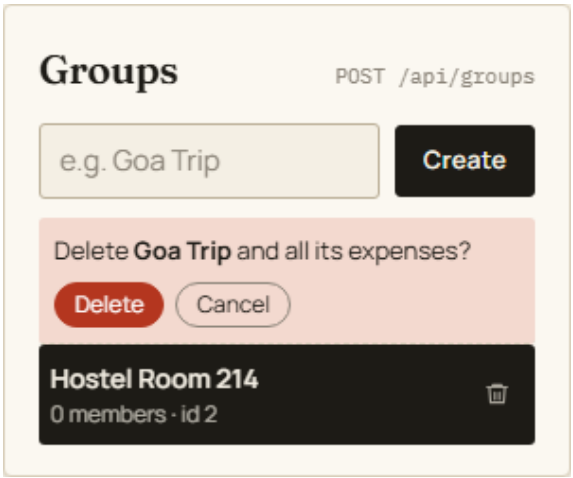


Figure 4.7 Inline confirmation before deleting a group

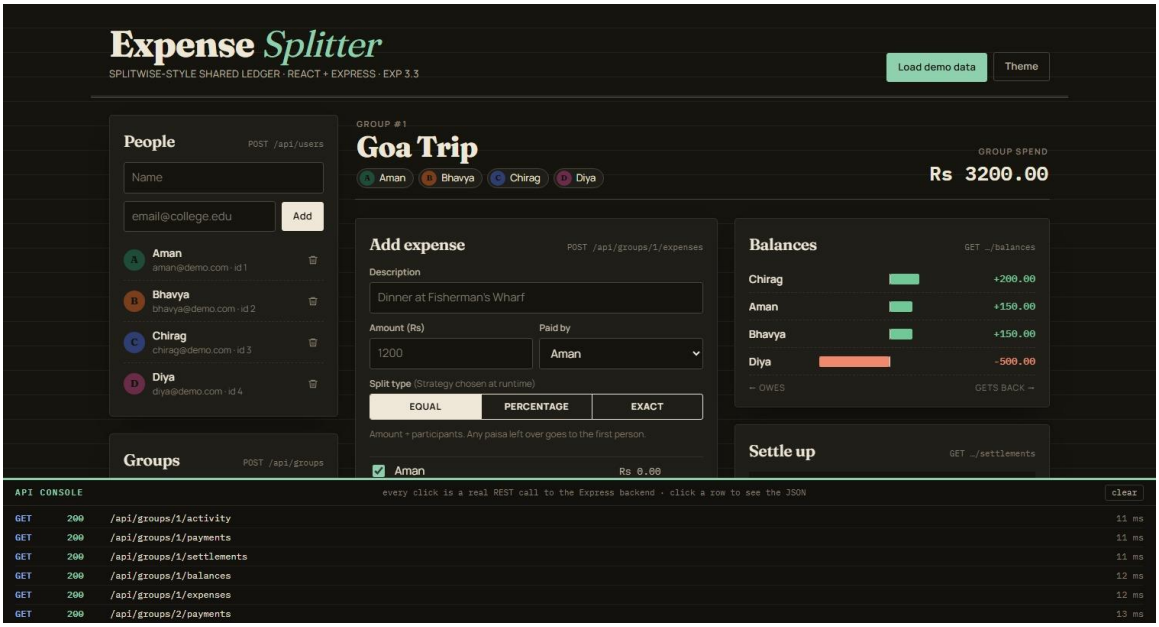


Figure 4.8 Dashboard in the dark theme

```
> npm test   (node --test, server/test/app.test.js)

ok full scenario gives expected balances and settlements (6.5099ms)
ok equal split puts the leftover paisa on the first person (0.4676ms)
> REST API
  ok create user returns 201 and the user (90.309ms)
  ok errors have the clean error body (7.1809ms)
  ok bad JSON, bad id, wrong method and unknown URL (15.2453ms)
  ok expense, filter, payment, undo and delete user conflict (34.6339ms)
ok REST API (153.5455ms)
i tests 6
i suites 1
i pass 6
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 597.477
```

Figure 4.9 Backend test suite run with npm test (node:test, 6 tests, 0 failures)

4.1.8. Testing and validation

The application was tested at three levels. A node:test suite (server/test/app.test.js) runs the complete scenario in code — balances, settlements, the rollback of an invalid expense and settle-up payments — and also starts the Express app on a random port to check status codes and error bodies over HTTP (Figure 4.9). A Postman collection of 41 requests with 59 assertions, organised in eight folders, exercises every endpoint, including all error paths; it saves IDs from each response into collection variables and deletes everything it created, so it can be re-run at any time. The collection was executed with Newman, Postman's

command-line runner (Figure 4.10). Finally, every screen of the React interface was used in both themes, and the screenshots in this chapter were captured automatically with Playwright. Table 4.5 summarises the functional test cases.

Table 4.5 Functional test cases and results

#	Test case	Expected result	Result
1	Run backend tests (npm test)	6 node:test tests pass	Pass
2	Build and start (npm run build, npm start)	Express on port 8080 serves the API and the React build	Pass (under 1 s)
3	Create 4 users, a group, add members	201 / 201 / 200; group has 4 members	Pass
4	EQUAL Rs 1200 among 4	Four shares of Rs 300.00	Pass
5	PERCENTAGE Rs 1000 at 25/25/30/20	Shares 250 / 250 / 300 / 200	Pass
6	EXACT Rs 1000 at 250/300/200/250	Shares as given	Pass
7	Balances	+400, +200, +150, −750; sum is zero	Pass
8	Settlements	Three payments, all from Diya	Pass
9	Percentages totalling 90	400; no expense saved (rollback)	Pass
10	EXACT total ≠ amount; unknown split type	400 with explanatory message	Pass
11	Duplicate e-mail	400 “Email already registered”	Pass
12	Unknown group or user ID	404 with JSON error body	Pass
13	Delete an expense twice	204, then 404; settlements recalculated	Pass
14	Record a payment, then pay more than owed	201, then 400 “only owes …”	Pass
15	Undo a payment twice	204, then 404	Pass
16	Delete a user who still has expenses	409 Conflict	Pass
17	Delete a group, then its users	204 for all; group then 404	Pass
18	Wrong HTTP method / malformed ID	405 / 400 with JSON body	Pass
19	Full Postman collection with Newman	41 requests, 59 assertions, 0 failures	Pass (6.2 s run)

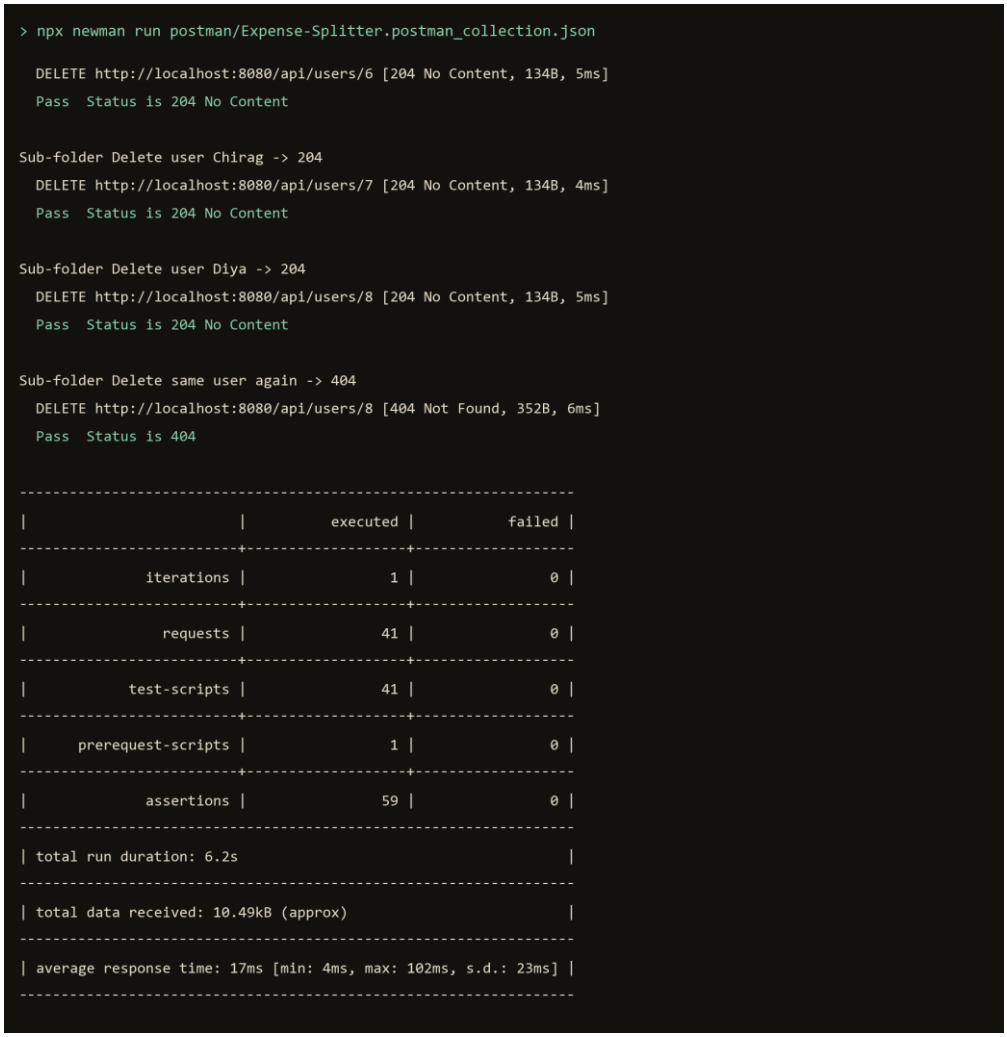


Figure 4.10 Newman run of the Postman collection (last two folders and summary)

Quality of the greedy settlement. Because the specification asked for the minimum number of transactions, the greedy algorithm was compared with an exact solver. The exact minimum equals n − k, where k is the largest number of disjoint groups of members whose balances sum to zero; this was computed by exhaustive search over subsets. Table 4.6 summarises the results for the project scenarios and for random groups with balances in multiples of Rs 50.

Table 4.6 Greedy settlement compared with the exact minimum

Case	Greedy payments	Minimum payments	Observation
Demonstration scenario	3	3	Optimal
After deleting the EXACT expense	3	3	Optimal
Balances +600, +400, +300, −800, −300, −200	5	4	Pairing +300/−300 first saves one payment
3,000 random groups of 4 members	100% optimal	—	Never worse
3,000 random groups of 5 members	97.8% optimal	—	At most 1 extra payment
3,000 random groups of 6 members	91.3% optimal	—	At most 1 extra payment
3,000 random groups of 7 members	81.3% optimal	—	At most 2 extra payments
1,500 random groups of 8 members	75.7% optimal	—	At most 2 extra payments

All functional test cases passed. The comparison confirms the literature [11]: the greedy method always stays within n − 1 payments and is optimal for typical small groups, but it is a heuristic, not a guarantee of the minimum. This is stated in the documentation and motivates the exact-settlement item in future work.

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1. Conclusion

Expense Splitter meets its objectives. It records group expenses with three interchangeable split strategies, saves each expense atomically, computes balances that always sum to zero, proposes a settlement of at most n − 1 payments, records partial settle-up payments with undo, and reports every error as clean JSON with the correct status code. The project visibly applies the concepts of full-stack development — a React component interface, a Node.js/Express REST API, layered services and repositories, unit-of-work rollback, central error middleware, modern JavaScript and the Strategy, Observer and Singleton patterns — and all 59 automated API assertions and the node:test suite pass.

The complete source code, Postman collection, README, Postman test guide and viva cheat-sheet are in the GitHub repository github.com/kaone31056789/expense-splitter-fsd. The app runs locally with npm run setup, npm run build and npm start, and is deployed on Vercel at https://expense-splitter-fsd.vercel.app.

Deviations from the expected results and their reasons:

- The specification asked for the *minimum* number of transactions. Testing showed that the greedy algorithm guarantees at most n − 1 payments but not always the minimum (Table 4.6), so the claim was corrected in the code comments and documentation.
- Express has no built-in 405 response and accepts any text as a route parameter. Every route therefore ends with an .all() handler that returns 405 with the allowed methods, and router.param() converts ids to numbers and returns 400 for values such as “abc”.
- A two-click confirmation on the expense delete button made users believe deleting did not work, and a stale list could show an expense already deleted elsewhere (404). Delete became one click with an automatic refresh; only the destructive group delete keeps an explicit confirmation.
- Deleting users and groups and recording settle-up payments were added after the original specification, which required new integrity rules (409 for users still in

expenses or payments, cascade for groups, and payments limited to the amount owed).

- On Vercel the Express app runs as a serverless function. Its memory is not shared between instances and is cleared when an instance stops, so the online demo can lose its data after a period of inactivity; locally, data lasts until the server stops. A hosted database is listed as future work.

5.2. Future Work

- **Exact settlement for small groups:** compute the true minimum by searching for the largest set of disjoint zero-sum subsets (dynamic programming over subsets) when $n \leq 15$, and keep the greedy method for larger groups.
- **Authentication and authorisation:** add sign-in with JWT checked by an Express middleware, so that each user sees only their own groups.
- **Integer money:** store all amounts as whole paise and convert to rupees only for display, removing floating-point rounding entirely.
- **UPI deep links:** generate upi://pay links or QR codes for each suggested payment and mark the payment as recorded once it is confirmed.
- **More expense features:** multi-currency trips with exchange rates, recurring expenses and receipt photos.
- **Real notifications:** e-mail or push notifications as additional Observer listeners.
- **Persistent database and CI:** replace the in-memory repositories with MongoDB or PostgreSQL behind the same repository interface, so data survives restarts and serverless cold starts; run npm test and Newman on every push with GitHub Actions.
- **Mobile client:** a React Native app that uses the same REST API.

REFERENCES

[1] National Payments Corporation of India, "Unified Payments Interface (UPI) – Product overview." [Online]. Available: https://www.npci.org.in/what-we-do/upi/product-overview

[2] Splitwise, Inc., "Splitwise." [Online]. Available: https://www.splitwise.com

[3] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[4] IEEE, *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Std 754-1985, 1985.

[5] Ecma International, ECMAScript Language Specification (ECMA-262). [Online]. Available: https://tc39.es/ecma262/

[6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[7] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[8] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.

[9] OpenJS Foundation, "Node.js documentation." [Online]. Available: https://nodejs.org/docs/latest/api/

[10] OpenJS Foundation, "Express – Node.js web application framework: API reference." [Online]. Available: https://expressjs.com/en/4x/api.html

[11] T. Verhoeff, "Settling multiple debts efficiently: An invitation to computing science," *Informatics in Education*, vol. 3, no. 1, pp. 105–126, 2004.

[12] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev/

[13] Vite, "Vite guide." [Online]. Available: https://vite.dev/guide/

[14] MDN Web Docs, "Using the Fetch API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

[15] OpenJS Foundation, "Node.js test runner (node:test)." [Online]. Available: https://nodejs.org/api/test.html

[16] Vercel, Inc., "Using Express.js with Vercel." [Online]. Available: https://vercel.com/docs/frameworks/backend/express

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[18] Postman, Inc., "Newman: the command-line collection runner for Postman." [Online]. Available: https://github.com/postmanlabs/newman

APPENDIX

A. Design Checklist

Table A.1 Design checklist

Item	Status	Where
Need justified with documented evidence	Done	Section 1.1
Problem stated without hinting at the solution	Done	Section 1.2
Literature reviewed with timeline and analysis	Done	Chapter 2
Measurable objectives defined	Done	Section 2.6
Constraints (cost, privacy, ethics, safety, standards) considered	Done	Section 3.2
At least two alternative designs compared	Done	Sections 3.4–3.5
Block diagram, ER diagram, flowchart and algorithm provided	Done	Section 3.6
Implementation shown with screenshots	Done	Section 4.1
Test cases executed and results recorded	Done	Tables 4.5–4.6
Source code, build files and test collection delivered	Done	Section 5.1

B. Project at a Glance

Table A.2 Technical summary of Expense Splitter

Item	Detail
Platform	Full-stack web application in the browser (Windows, macOS, Linux); deployed on Vercel
Language / framework	JavaScript; React 18 (Vite) frontend, Node.js + Express 4 backend
Persistence	In-memory repositories with IDENTITY-style ids and an undo log for rollback
Build / run	npm run setup · npm run build · npm start → http://localhost:8080 (npm run dev for hot reload)
Architecture	Controller–service–repository; 34 backend modules (~1,420 lines), 8 React modules (~820 lines)
Design patterns	Strategy (split types), Observer (expense events), Singleton (ConfigService)
API	18 REST endpoints; status codes 200, 201, 204, 400, 404, 405, 409
Money handling	Half-up rounding to 2 decimals on the decimal text; remainder assigned to one participant
Settlement	Greedy over two binary-heap priority queues; at most n − 1 payments; O(n log n)
Testing	node:test unit + HTTP tests; Postman collection of 41 requests and 59 assertions (Newman)

C. Key Source Code Excerpts

(a) Strategy objects kept in a Map and chosen at run time — ExpenseService.js

```
const strategies = new Map([
  ['EQUAL', new EqualSplitStrategy()],
  ['EXACT', new ExactSplitStrategy()],
  ['PERCENTAGE', new PercentageSplitStrategy()]
]);

const strategy = strategies.get(splitType);   // "EQUAL", "PERCENTAGE" or "EXACT"
if (!strategy) {
  throw new InvalidSplitException(`Unknown split type '${request.splitType}'`);
}
```

(b) All-or-nothing expense creation — ExpenseService.js and Repository.js

```
addExpense(groupId, request) {
  return transactional(() => {
    ...
    const expense = expenseRepository.save(new Expense(...)); // 1. save
    const shares = strategy.calculateShares(amount, splits); // 2. may throw
    for (const [userId, share] of shares) {
      expense.addShare(new ExpenseShare(userService.getUser(userId), share));
    }
    expenseEventPublisher.notifyExpenseAdded(expense);      // 3. observers
    return expense;
  });
}

export function transactional(fn) {
  currentTransaction = [];       // undo log, filled by save() and delete()
  try {
    return fn();
  } catch (err) {
    currentTransaction.reverse().forEach((undo) => undo());   // ROLLBACK
    throw err;
  } finally {
    currentTransaction = null;
  }
}
```

(c) Net balances with reduce and flatMap — BalanceService.js

```
const sumBy = (items, keyOf, valueOf) => items.reduce((totals, item) => {
  totals.set(keyOf(item), (totals.get(keyOf(item)) ?? 0) + valueOf(item));
  return totals;
}, new Map());

const paid = sumBy(expenses, (e) => e.paidBy.id, (e) => e.amount);
const owed = sumBy(expenses.flatMap((e) => e.shares),
                   (s) => s.user.id, (s) => s.shareAmount);
```

(d) Greedy settlement with two max-heaps — BalanceService.js

```
const creditors = new PriorityQueue((a, b) => b.amount - a.amount);
const debtors   = new PriorityQueue((a, b) => b.amount - a.amount);
while (!creditors.isEmpty() && !debtors.isEmpty()) {
  const creditor = creditors.poll();
  const debtor = debtors.poll();
  const pay = config.round(Math.min(creditor.amount, debtor.amount));
  const message = `${debtor.name} pays ${creditor.name} ${config.format(pay)}`;
  result.push({ ..., amount: pay, message });
```

```
  creditor.amount = config.round(creditor.amount - pay);
  debtor.amount = config.round(debtor.amount - pay);
  if (creditor.amount > config.tolerance) creditors.add(creditor);
  if (debtor.amount > config.tolerance) debtors.add(debtor);
}
```

(e) Singleton with a private constructor token — ConfigService.js

```
const TOKEN = Symbol('ConfigService');
export class ConfigService {
  static #instance = null;
  constructor(token) {
    if (token !== TOKEN) throw new Error('Use getInstance()');   // "private"
    ...
  }
  static getInstance() {
    if (!ConfigService.#instance) {
      ConfigService.#instance = new ConfigService(TOKEN);
    }
    return ConfigService.#instance;
  }
}
```

D. Sample API Requests and Responses

(a) Add a PERCENTAGE expense — POST /api/groups/1/expenses

```
{
  "description": "Cab to airport", "amount": 1000, "paidBy": 2,
  "splitType": "PERCENTAGE",
  "splits": [ {"userId": 1, "value": 25}, {"userId": 2, "value": 25},
              {"userId": 3, "value": 30}, {"userId": 4, "value": 20} ]
}
```

(b) Error response when the percentages do not total 100 — 400 Bad Request

```
{ "timestamp": "2026-10-28T12:26:41.392", "status": 400, "error": "Bad Request",
  "message": "Percentages must add up to 100, but got 90" }
```

(c) Settlement for the demonstration scenario — GET /api/groups/1/settlements → 200 OK

```
[ { "fromUser": "Diya", "toUser": "Aman", "amount": 400,
    "message": "Diya pays Aman Rs 400.00" },
  { "fromUser": "Diya", "toUser": "Chirag", "amount": 200,
    "message": "Diya pays Chirag Rs 200.00" },
  { "fromUser": "Diya", "toUser": "Bhavya", "amount": 150,
    "message": "Diya pays Bhavya Rs 150.00" } ]
```

(d) Record a partial settle-up payment — POST /api/groups/1/payments → 201 Created

```
{ "fromUserId": 4, "toUserId": 1, "amount": 250 }
```

EXPENSE SPLITTER: A SPLITWISE-STYLE GROUP EXPENSE SETTLEMENT SYSTEM USING REACT, NODE.JS AND DESIGN PATTERNS

Submitted by

Parikshit (24BET10298)
Dhruv Gouniyal (24BET10295)
Sagar Makkar (24BET10296)
Manan (24BET10199)

Under the supervision of

Suchi Sharma, Assistant Professor
Department of Information Technology, Chandigarh University

Live: expense-splitter-fsd.vercel.app · GitHub: github.com/kaone31056789/expense-splitter-fsd

— End of Report —